Kalshi Market Lab

Research, mathematics and engineering audit

November 4, 2026 | America/New_York

Public research and paper trading at https://predictir.dev

A source-backed account of the hypotheses tested, models trained, code shipped, costs considered and evidence still missing. This report covers independent project engineering and excludes employment verification.

Evidence class	Meaning
Implemented	Code exists and is exercised by named tests or reproducible tools.
Measured	A dated local fixture or held-out model result, with its scope stated.
Experimental	A usable paper-research estimate whose statistical promotion gate has not passed.
Not established	No claim is made for unavailable data, private trader motives or realized trading returns.

The notable mathematical addition is a tested composition of calibrated independent forecasts, fresh score-clock conditioning, executable depth, uncertainty-aware position decisions, a discrete risk constraint and an event-overlap graph. Established papers motivate the components; the project's own measurements evaluate this implementation.

Source revision available when the report was rendered: e1a26780bd03fb82b4e27dd4d6adadfc8d77a508

Measured model comparisons

Pregame: lower Brier is better, same 514-game holdout

The 1.554% relative Brier point improvement is observable. Its paired whole-week uncertainty interval includes zero, so the challenger remains an explicit experimental option rather than being silently promoted. The neural candidate was trained and its non-convergence retained in the record.

In-play: separate 20-game / 60-quarter-state replay

The live replay is small and retrospective, with three correlated quarter states per game. Forty earlier calibration games estimate the score-process scale; twenty held-out games evaluate it. This does not establish a calibrated live-money advantage. The model also requires a current matched score and clock before it emits an in-play estimate.

In-play calculation

P(home win | margin L, elapsed fraction t) = Phi((L + mu*(1-t)) / (sigma*sqrt(1-t))). The independent pregame chance p0 supplies mu=sigma*Phi_inverse(p0). Sigma is estimated from historical calibration-game score increments. Current fitted sigma is 13.3579 points, with a whole-game bootstrap sensitivity range [11.4684, 15.1933].

This regulation model omits possession, strategic fouling, live lineups and non-Gaussian endgame behavior. Missing scores are not zero; missing time is not inferred from the scheduled start. Overtime and unsupported boundaries are withheld. Current game state expires after 30 seconds; research caches use five seconds for in-play estimates. Source: Stern [1]; exact capture and replay files accompany the code.

Engineering measurements

Operation	Measured result	Interpretation
Compiled ensemble	1.056 microseconds per cached score	Local CPU inference; excludes sources, network and rendering.
Repeated fill sizing	0.295 ms including construction vs 182.996 ms	1,000 sizing queries on one synthetic 1,000-level book; cash-flow parity checked.
Book broadcast	26,699 to 1,390 bytes	Bounds displayed depth; retains full indexed source reconstruction and fresh REST execution.
Schema and fanout	See runtime-benchmark.json and RUNTIME_RESEARCH.md	Contains query plans, concurrency, backpressure and negative partition results.

The sizing curve parses and sorts the book once, builds quantity/notional/fee prefixes, then answers a quantity query by binary search. Construction is O(L log L) including sorting plus O(L) prefix work; subsequent fill queries are O(log L). This saves repeated parsing during sizing and risk enumeration. It is not a claim that the whole website became hundreds of times faster.

Feature-by-feature audit

Reviewed November 4, 2026 (America/New_York). The full requested feature list is tracked below. Implemented features use this repository and the existing Vercel, Railway and Supabase accounts. No new paid integration was purchased. A local benchmark is not an end-to-end latency or trading-return result.

Forecasts, position decisions and portfolio mathematics

The default NBA probability remains the regularized team-strength model. A separately selectable challenger trains ridge logistic, depth-two XGBoost and a 16-by-8 neural network on actual historical scores. A selection partition chooses the classifier/blend, a separate partition fits beta calibration, and a final chronological partition evaluates it. The selected composition is 75% team strength and 25% lag-feature logistic before beta calibration. The neural network's non-convergence is recorded rather than silently discarded.

Across 514 held-out games, Brier error decreases from 0.2025336534 to 0.1993854751, a 1.554% relative point improvement. Log loss decreases from 0.5935887721 to 0.5860446437. The paired weekly 95% improvement interval is approximately [-0.0011701, 0.0075196]. This promising experiment is usable through the experimental model setting; its formal promotion gate remains false. The held-out results concern outcome forecasts, not historical entries, fills or realized returns. See `NBA_RESEARCH.md`, `nba-challenger-validation.json` and `nba-inference-benchmark.json`.

The experimental in-play NBA model conditions on an actual current score and clock, following Stern's score-difference diffusion. Its prior comes from an independent pregame model. Its score-variance scale is fitted from historical calibration games, not invented. It refuses unsupported overtime and missing/stale live state. `NBA_INPLAY.md` and `nba-inplay-validation.json` record the historical state replay, parameter estimation and limitations.

The project-specific mathematical composition is an auditable combination of established methods, not a claim to a newly proved theorem. The new elements in this application are their integration with exact event mapping, dated model residuals, current executable depth, explicit uncertainty and risk constraints, and the known-event exposure graph.

For a selected side with independent chance p, range [l,u], quantity q and all-in entry cost C(q), expected settlement profit is p*q-C(q). Its sensitivity range is [l*q-C(q),u*q-C(q)]. If B is the entered budget, winning and losing ending values are B-C(q)+q and B-C(q). The chance of reaching 2B is calculated from these two actual branches; if neither branch reaches the target, it is zero. This is a settlement calculation, not a fabricated price-path forecast.

Budget scenarios spend only what the current book and fees permit. A separate discrete Kelly optimizer maximizes the lower-bound expected log growth subject to a drawdown-moment condition. With lambda=log(beta)/log(alpha), it checks l*(W_win/B)^(-lambda)+(1-l)*(W_lose/B)^(-lambda)<=1. The displayed defaults alpha=0.8 and beta=0.1 are explicit risk-policy assumptions. A repeated-wealth guarantee would require correct stationary probabilities and repeated independent opportunities; those conditions are not claimed for this app.

The live paper tracker compares q*p with current net bid proceeds R(q). HOLD requires the lower value q*l to exceed R(q) plus a visible action buffer. EXIT requires even q*u to fall below current proceeds minus the buffer. Overlap produces REVIEW. TRIM applies an explicitly entered position-loss limit. A falling quote alone is not evidence that the original thesis failed. Partial depth, stale quotes or missing model evidence withhold full-position P/L or actions. Confirmed binary settlements are recorded separately from bid closes. Multiple open lots share one proportional allocation of their common bid book, with separately rounded lot fees; the portfolio cannot reuse the best bid or count unavailable liquidity twice. Wrong-contract packets, failed refreshes and pregame estimates after event start cannot issue current model actions.

Validated rankings require a positive lower-bound edge after the budget-sized fill and buffer. Experimental rankings allow positive point-estimate edges while showing the lower-bound risk and failed model-promotion gate. Every row uses the same entered budget as an alternative scenario; the app does not pretend that each row creates additional portfolio cash. The event-overlap graph prevents duplicate ranked exposure to a known common event. Unknown cross-event correlations remain unknown.

Complete feature disposition

Requested item	Implemented or evaluated result	Cost and benefit basis
Enums	Native PostgreSQL job-transition state enum; application model/theme/action inputs are bounded allowed sets.	Rejects invalid state transitions without another service.
Safe deletion, object drops and cascade deletes	Forward migration preview, explicit apply, hash drift checks, advisory locking and destructive-DDL rejection; immutable evidence/journal guards; no production deletion or drop was performed.	Prevents accidental destructive operations and preserves provenance. Cascades are not introduced for financial evidence.
Edge Functions	Existing Vercel API functions provide the same-origin execution layer. Supabase Edge Functions were priced and evaluated as an additional hop.	A second execution service is not provisioned where the same operation is already implemented in code. Region colocation is measured separately.
Debugging and general logger	Structured request completion, duration, status, trace ID and stream health; credential/body/query redaction.	Operational evidence without paying for an additional log product or logging private payloads.
W3C Trace Context	Validated traceparent propagation and new spans; controlled tracestate handling; untrusted baggage is dropped.	Correlates frontend/backend requests while limiting metadata leakage.

Requested item	Implemented or evaluated result	Cost and benefit basis
Realtime Protocol and low-latency Broadcast	Shared native exchange WebSocket, correct control-message sequencing, subscription cleanup, bounded snapshots and one multiplexed portfolio stream.	One upstream feed fans out to multiple clients; no duplicate managed feed subscription.
Presence	Same-origin ledger coordination and browser-tab update mechanisms provide the needed local synchronization. Public online-user tracking is not added.	Supports this personal paper workflow without sending private activity to a presence provider.
Postgres Changes and triggers	Semantic job-state transitions generate an append-only typed outbox and lightweight notifications; listener wakeups complement polling recovery.	Notifications follow changes, not every quote or health request. No public private-table replication channel is enabled.
Scalability benchmarks	Reproducible model, sizing, stream fanout, database and partition/projection experiments; bounded capacities and backpressure.	Measured workload fixtures, not an unmeasured claim of HRT-scale infrastructure.
Trade tracking	Live executable marks, cost basis, fees, realized/unrealized P/L, hold/trim/exit/review decisions and explicit confirmed settlement.	Works from actual public data; no real-order route.
Recommended trades and price ranges	Budget input, independent model chances, payoff branches, target probability, cost ceiling, risk size and an event-overlap graph.	Mathematics runs in application code; language-model tokens are not used to calculate prices or odds.
Cheapest MCP	Existing read-only MCP server plus the current low-cost gateway model; selection follows the website's chosen research model.	Current catalog pricing and subscription-credit eligibility are documented; no extra MCP subscription.
GraphQL	A projected/nested read comparison is benchmarked; active search returns nested source provenance in one SQL query.	No extra GraphQL gateway or parser is purchased merely to duplicate a bounded query.
Graph algorithms and graph data engineering	Known-event exposure graph, deduplication and relational source/event/contract/observation lineage.	Captures known dependencies without purchasing a graph database or inventing unknown correlations.
Globally distributed Realtime	Vercel CDN distribution and regional backend deployment are assessed; the maintained market feed has one authoritative region.	A second paid regional feed is not claimed as deployed. Colocation and fanout benchmarks quantify the current tradeoff.

Requested item	Implemented or evaluated result	Cost and benefit basis
Flutter	Web interface supports desktop/mobile layouts without a second native codebase.	Flutter remains a costed client-port option; it does not improve the current browser runtime by itself.
Local Development and CLI v3	Pinned public Node/Python manifests, reproducible frozen data, disposable PostgreSQL tests and migration preview/apply tooling.	Stable commands remain the release path; experimental CLI commands are inspected before use.
Experimental supabase stack commands	Supabase CLI 3.0.0 and its experimental help were inspected; the named stack subcommand was not listed in that installed inventory.	No unverified experimental stack deployment is claimed. Stable local start and migration/test commands remain independently runnable.
pgTAP	Actual SQL assertions validate schema policies, transition notifications and immutable-record behavior.	Tests run on a disposable database; production data is not deleted to test deletion guards.
pg-delta and migra	Schema drift and destructive DDL risks are covered by migration hashes and reviewed additive SQL.	Blind generated diffs are not applied to the hosted financial-evidence schema.
Seeding historical data	Frozen real NBA game captures and source hashes reproduce training; synthetic test fixtures are labeled separately.	Real historical scores are not relabeled as prospective predictions or trade fills.
Full Text Search	Generated text-search document, GIN index, bounded parameterized websearch query, latest-per-contract results and source joins.	Active retained-history search; selecting a result requests fresh contract context.
Webhooks	Internal outbox/notification wakeups and same-origin browser update events serve current change propagation.	No public webhook secret or paid webhook relay is needed for this internal pipeline.
Partitioning	Local flat-versus-time-partitioned query/pruning experiment and growth thresholds.	Partitioning existing small tables is not claimed as deployed; benchmark evidence guides a later additive migration.
pgloader import	The current typed JSON/SQL ingestion validates provenance and supports batch inserts.	pgloader is reserved for an actual external database import; there is no external legacy database to migrate.
XGBoost, neural networks and ML	Actual trained ridge, XGBoost and neural candidates, chronological comparisons, calibration, compiled inference and parity tests.	Training runs locally; production does not need Python, a GPU instance or a managed ML endpoint.

Requested item	Implemented or evaluated result	Cost and benefit basis
Joins and nested tables	Parameterized source joins produce nested provenance for retained observations.	One database query replaces a client-side N+1 source lookup.
jsonb and arrays	Typed financial columns remain separate from structured metadata; JSON batch ingestion and quality-flag arrays have explicit grain checks.	Avoids an indiscriminate JSONB/GIN index over every field.
Indexes, B-tree, binary tree, BRIN and GIN	Workload-specific ordered lookup, compact time-range index and full-text index; prefix fill curves and indexed books serve numerical queries.	EXPLAIN and synthetic-scale benchmarks determine selection. No blanket indexing or routine production REINDEX.
Index advisor and database statistics	Read-only catalog/statistics, index-use and query-plan inspection plus reproducible local tests.	Does not assume that a recently created index's low usage proves it is unnecessary.
RLS	Application schema and new tables remain isolated from anonymous roles; explicit backend grants and owner-token checks continue.	No private journal data is added to public realtime, search or AI tools.
WHERE, ORDER BY, LIMIT and EXPLAIN	Bounded queries, latest-row selection, proper ordering and explicit source/time predicates.	Plans are inspected at representative synthetic scale and compared with actual small production statistics.
npm, pnpm, yarn and Homebrew	npm's pinned lockfile is the supported release install; alternative package managers and Windows package installation are documented options.	Multiple conflicting lockfiles or macOS Homebrew do not improve this Windows release's reproducibility.
Black futuristic UI and layout options	Graphite, Midnight and Light; comfortable/compact density and layout settings; visible model mode and readable controls.	Reuses the app's design language and requires no paid design service.
Top public traders, all-time/week/four months	Provenance-checked public closed-trade analysis tool computes observed-sample top-30 windows, fees, concentration and entry-dollar return.	Official public leaderboard retrieval was throttled; a complete verified top-30 history and trader motives are not invented.

Public trader research and source limits

Kalshi's official help describes an opt-in leaderboard. Public leaderboard pages show projected profit rankings, which differ from verified account-wide realized return. The public trade API does not establish every participant's identity, capital, hedges or private positions. The direct public leaderboard request returned HTTP 429 during this audit; the access limit was respected. No private profiles, messages or contacts were accessed.

The supplied-data tool `node tools/analyze_public_traders.mjs public-closed-trades.json analysis.json` accepts only explicitly public, source-linked, deduplicated closed-trade cash flows. It calculates all observed history, trailing seven days and trailing four calendar months. Fewer than 30 available traders produce fewer than 30 rows. Missing records never become zero trades. Public winner anecdotes and forum copy-trading claims suggest tests for concentration, specialization, latency and entry cost; they are not training labels or proof of a person's strategy.

Cost and integration rule

The self-built calculation engine, MCP server, stream fanout, job queue, FTS search and logger run on the existing stack. The dated gateway catalog lists Gemini 2.5 Flash-Lite at $0.10/M uncached input tokens and $0.40/M output; DeepSeek V4.1 Flash at $0.195/M and $0.78/M; GPT-5 nano at $0.05/M and $0.40/M. Provider availability, regional routing and tool quality still require live verification. Gemini remains the currently verified route. Model computation does not use an LLM.

Google AI Pro can include a claimed Google Developer Program cloud-credit benefit, and eligible ChatGPT Pro 100 subscriptions can activate API credits for a linked organization. These are account benefits, not credentials that the website can automatically borrow from a signed-in consumer chat session. No unverified credit balance is included in this app's operating forecast. Existing app and platform quotas remain active. The report distinguishes installed features, tested local experiments, live verification and future optional provisioning.

The bounded live Gateway comparison made exactly four requests: one scoped tool-routing step and one grounded follow-up per model. Both Gemini 2.5 Flash-Lite and GPT-5 nano passed both tasks. Gemini used 487 input and 123 output tokens, took 2.151 seconds across the two steps and cost $0.00009790; nano with minimal reasoning used 799 input and 179 output tokens, took 2.745 seconds and cost $0.00011155. Gateway-reported costs agree with the recorded token rates. Gemini remains the route because nano was 13.94% costlier and 27.58% slower in this tiny sample. This is not a general quality or p95 comparison and excludes remote MCP transport. `ai-gateway-benchmark.json` and the opt-in benchmark script preserve the exact requests, grading and costs.

Ranking the requested integration surfaces

Priority	Surface	Benefit and implementation decision
1	Observability	Request traces, redacted structured logs, source-age checks, database diagnostics and usage allowances expose correctness and latency problems immediately. These are implemented in the existing processes first; additional paid retention or analytics products require a measured gap.

Priority	Surface	Benefit and implementation decision
2	AI Gateway	The existing route supplies the read-only research assistant. A $3 monthly project budget, atomic application reservations and metered refunds bound usage; the actual two-model comparison above informs the current choice.
3	Stores	Existing Supabase PostgreSQL and private object storage provide relational state, queues, full-text search and evidence archives. A second managed database/cache must beat the existing indexed/in-process path on the same workload before adding its bill and consistency work.
4	Marketplace wrappers	Provider-specific installation and secret provisioning can save integration work. Current Kalshi, ESPN, Spotify, X and MCP adapters already run in code; no additional paid wrapper was purchased. Compare a wrapper's exact capability, latency, rate limits and price against those adapters before adopting it.
5	Supabase OAuth integration	This guide builds an integration that other Supabase users authorize against their own projects. It becomes useful for a future multi-tenant product or external-project connector. The current app's user/private-journal boundaries and backend credentials are implemented directly; no broad account-management grant is introduced for this deployment.

Small predicted improvements remain experiment candidates. The partitioning experiment is an example of testing the hypothesis and keeping its negative result: the indexed heap was faster for that fixture. The row above records the present implementation and the evidence needed for a later replacement, rather than treating a product name as a performance result.

The existing single Railway backend was moved from SFO to Virginia without adding another service or volume. Seven pooled SELECT 1 samples per location measured mean database round-trip time of 55.30 ms before and 19.26 ms after, a 36.04 ms / 65.18% reduction in this short comparison. Applied configuration, the successful deployment manifest and a running instance confirm the destination region. This is a database-path measurement, not a whole-app speed claim. `deployment-latency.json` and `RUNTIME_RESEARCH.md` preserve the evidence and scope.

Live release checks exercised a $20-to-$30 scenario change, opening the matching paper ticket, a one-contract simulated buy, executable bid marking, an explicit risk-limit response and a close. The observed 18-cent entry and 15-cent net close produced a 3-cent realized paper loss, including modeled fees. Visible opinions and rankings refresh while the page is visible; expired sources, old quotes and failed refreshes become STALE and withhold old numerical projections. Decimal-second NBA clocks are parsed without confusing seconds and minutes; conflicting provider clock fields withhold an estimate. A real completed assistant call reserved $0.025, recorded a conservative $0.001 usage charge and refunded $0.024. Read-only hosted checks confirmed private-schema isolation and runtime grants after migration. These checks are engineering observations, not a trading-performance evaluation.

Recruiter-facing claim wording

- Built a public prediction-market research and paper-trading application with shared WebSocket ingestion, bounded multiplexed portfolio updates, transactional PostgreSQL jobs, RLS, source lineage and immutable compressed evidence archives.

- Trained and compared ridge, XGBoost and neural NBA models on 2,569 real historical games; exported a calibrated ensemble to JavaScript with native-inference parity, chronological model selection and auditable residual-based sensitivity.

- Implemented fee- and depth-aware scenario sizing, discrete risk-constrained log-growth allocation, live paper-position decisions and confirmed settlement accounting; separated research estimates from realized paper results.

Use benchmark numbers only with their exact fixture and scope. Do not turn the 1.554% retrospective Brier point improvement into a general accuracy, profitability or return claim. No employment verification is part of this work.

Audit and reproduction checklist

Inspect the frozen source hashes and split dates; reproduce native model training; verify Python/JavaScript parity; run the PostgreSQL and pgTAP checks; replay the independent cash-flow tests; review the full runtime query plans; compare hosted latency before/after region changes; and inspect a paper entry, live mark, close and confirmed settlement. Use the deployment guide for private environment setup. Credentials and private journals are excluded from this report.

Pinned PDF regeneration: create a Python environment, install tools/report-requirements.txt, then run python tools/build_research_audit.py --output /chosen/path/Research_Audit.pdf. No bundled private runtime or proprietary document package is required.

Research sources and what they contribute

[1]-[6] supply score-process, allocation, calibration, tree/NN and proper-score methods. [7]-[11] supply first-party descriptions of quantitative research and systems practice; they do not disclose proprietary strategies or certify this app. [12]-[25] support implementation and pricing decisions. [26]-[28] support the public-trader evidence boundaries and hypotheses. Forum claims remain unverified and do not train the outcome models.

The broad search included first-party research-firm material, original papers, platform documentation and public community discussion. The report does not infer a top trader's private tools or claim access to proprietary HRT methods. A direct official leaderboard request was throttled; no rate-limit bypass or private-profile retrieval was used.

[1] Stern: Brownian score-difference model

[2] Busseti, Ryu and Boyd: Risk-Constrained Kelly Gambling

[3] Kull, Silva Filho and Flach: Beta calibration

[4] Chen and Guestrin: XGBoost

[5] Guo et al.: On Calibration of Modern Neural Networks

[6] Gneiting and Raftery: Strictly Proper Scoring Rules

[7] HRT: Machine Learning and AI

[8] HRT: SharedWorker framework for real-time trading views

[9] HRT: Engineering, latency and research scale

[10] Two Sigma: Scientific research and technology approach

[11] Citadel: Alternative data research workflow

[12] PostgreSQL 17: Index types

[13] PostgreSQL 17: Partitioning

[14] W3C Trace Context

[15] Supabase Realtime source

[16] Supabase Broadcast

[17] Supabase Presence

[18] Supabase Postgres Changes

[19] Supabase Edge Function pricing

[20] Railway deployment regions

[21] Railway resource pricing

[22] Vercel AI Gateway live model catalog

[23] Google Developer Program credit benefits

[24] Eligible ChatGPT API-credit benefit

[25] Vercel: ChatGPT API credits integration

[26] Kalshi official leaderboard explanation

[27] Kalshi public leaderboard

[28] Community winner-sample discussion, unverified hypotheses